1️⃣ What is Java?

Answer:
Java is a **high-level programming language** that is **object-oriented**, **platform independent**, and **robust**.
It allows us to write code once and run anywhere because Java code is compiled into **bytecode**, which runs on **JVM**.
As a tester, Java is important because **most automation tools like Selenium, TestNG, JMeter** are based on Java.

2️⃣ What is JVM (Java Virtual Machine)?

Answer:
JVM is a **virtual machine** that executes Java bytecode.
It converts **bytecode into machine-specific code** and allows Java programs to run on any platform.
JVM also handles **memory management**, **garbage collection**, and **security**.

3️⃣ What is JRE (Java Runtime Environment)?

Answer:
JRE provides the **environment to run Java programs**.
It contains **JVM + core libraries**.
If we only want to **run Java programs**, JRE is enough.
It does **not contain compiler**, so we cannot write or compile code using JRE.

4️⃣ What is JDK (Java Development Kit)?

Answer:
JDK is used to **develop Java applications**.
It contains **JRE + compiler + debugging tools + development libraries**.
Without JDK, we cannot write or compile Java programs.

5️⃣ Difference between JVM, JRE, and JDK

Answer:

- **JVM:** Runs Java bytecode, handles execution.
- **JRE:** Provides environment to run Java programs (JVM + libraries).
- **JDK:** Provides tools to **develop** Java programs (JRE + compiler + tools).

6️⃣ How to create a Java project in Eclipse?

Answer:

1. File → New → Java Project → Give project name → Finish
2. Project → New → Class → Give class name (starts with uppercase) → Finish

Note: Execution always starts from **main()** method.

Example:

```
public class FirstJavaProgram {
    public static void main(String args[]) {
        System.out.println("Hello Java");
    }
}
```

7️⃣ What is a Class?

Answer:
Class is a **collection of variables and methods**.
It is a **logical entity** (exists only in code).
Class **cannot store anything** unless we create an **object** of it.

8️⃣ What is an Object?

Answer:
An object is an **instance of a class**.
It is a **physical entity** and occupies memory.
Objects communicate with each other using **methods**.

Example:

```
class Student {
    int studentId;
    String name;
    void display() {
        System.out.println(studentId + " " + name);
    }
}

public class TestStudent {
    public static void main(String args[]) {
        Student s1 = new Student();
        s1.studentId = 101;
        s1.name = "Rahul";
        s1.display();
    }
}
```

9️⃣ What is the main() method?

Answer:
main() is the **entry point of Java program**.
Program execution starts from **main()**.
Syntax:

```
public static void main(String[] args) {
    // program starts here
}
```

🔟 What are Variables in Java?

Answer:
Variables are **containers that hold values**.
We can **declare** and **initialize** them.

Examples:

```
int x; // declaration
x = 100; // initialization
```

```
int y = 10;
float f = 10.5f;
double d = 10.555;
char c = 'y';
boolean flag = true;
```

1️⃣1️⃣ What are Data Types in Java?

Answer:
Data types define **what type of data a variable can store**.

1️⃣ **Primitive Data Types:** int, double, char, boolean, short, float
2️⃣ **Derived / Non-Primitive Data Types:** Array, ArrayList, String, HashMap

Non-primitive types are **predefined classes** in Java.

1️⃣2️⃣ What are Operators in Java?

Answer:
Operators perform operations on variables and values.

Types:

- Arithmetic: + - * / %
- Relational: == != < > <= >=
- Logical: && || !
- Increment/Decrement: ++ --
- Assignment: =

1️⃣3️⃣ What are Conditional Statements?

Answer:
Conditional statements **control the program flow based on conditions**.

Types:

- if
- if–else
- nested if–else
- switch-case

Example:

```
if(score > 50) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

1️⃣4️⃣ What are Loops in Java?

Answer:
Loops are used to **repeat a block of code** multiple times.

Types:

- while loop
- do–while loop
- for loop

Example:

```
for(int i=0; i<5; i++) {
    System.out.println(i);
}
```

1️⃣5️⃣ What are Jump Statements?

Answer:
Jump statements are used to **control loop execution**.

- `break` → exits loop or switch
- `continue` → skips current iteration

1️⃣6️⃣ What is an Array?

Answer:
Array is a **collection of elements of the same data type**.

Example:

```
int a[] = new int[5];
a[0] = 100;
a[1] = 200;
```

- Size: `a.length`
- Access element: `a[2]`
- Read all values:

```
for(int i : a) System.out.println(i);
```

1️⃣7️⃣ What is a Two-Dimensional Array?

Answer:
Two-Dimensional Array stores data in **rows and columns**.

Example:

```
int a[][] = new int[3][2];
a[0][0] = 100; a[0][1] = 200;
a[1][0] = 300; a[1][1] = 400;
```

- Rows: `a.length`
- Columns: `a[0].length`

1️⃣7️⃣ What is String in Java?

Answer:
String is a **non-primitive data type** and already an **object** in Java.
String is **immutable**, which means once the object is created, its value **cannot be changed**.
If we modify a String, a **new object** is created in memory.

Example:

```
String s = "Welcome";
```

1️⃣8️⃣ Why String is immutable?

Answer:
String is immutable for **security**, **memory optimization**, and **performance**.
In applications like **database connections, URLs, credentials**, immutability prevents unwanted changes.
Also, Java uses **String Constant Pool**, which saves memory.

1️⃣9️⃣ Difference between String, StringBuilder, and StringBuffer

Answer:

- **String:** Immutable, thread-safe, slow for modifications
- **StringBuilder:** Mutable, not thread-safe, fast
- **StringBuffer:** Mutable, thread-safe, slower than StringBuilder

Use case:

- String → constants
- StringBuilder → single-threaded applications
- StringBuffer → multi-threaded applications

2️⃣0️⃣ What is OOP?

Answer:
OOP (Object Oriented Programming) is a programming approach where we design programs using **classes and objects**, similar to real-world entities.
It improves **code reusability**, **readability**, **maintenance**, and **security**.

2️⃣1️⃣ What is Encapsulation?

Answer:
Encapsulation is the process of **wrapping data (variables) and methods into a single unit (class)** and restricting direct access to variables.
All variables should be **private**, and access should be through **getter and setter methods**.
It provides **data hiding and security**.

2️⃣2️⃣ What is Inheritance?

Answer:
Inheritance allows one class to **acquire properties and behavior of another class**.
It creates an **IS-A relationship** and promotes **code reusability**.

Example:

```
class Parent {}
class Child extends Parent {}
```

2️⃣3️⃣ Types of Inheritance in Java

Answer:

- Single
- Multilevel
- Hierarchical

⚠ Multiple and Hybrid inheritance are **not supported using classes**, but supported via **interfaces**.

---

24 What is Polymorphism?

Answer:
Polymorphism allows **one method or object to behave differently in different situations**.
It improves **flexibility and scalability** of code.

---

25 What is Method Overloading?

Answer:
If a class has **multiple methods with the same name but different parameters**, it is called method overloading.
It is **compile-time polymorphism**.

Rules:

- Same method name
- Different number or type of parameters

---

26 What is Method Overriding?

Answer:
If a child class provides its own implementation of a method already defined in parent class, it is called method overriding.
It is **run-time polymorphism**.
Method signature must be **exactly same**, but body is different.

---

27 Difference between Overloading and Overriding

Answer:

- Overloading → same class, compile-time, parameters change
- Overriding → parent-child classes, runtime, method body changes

---

28 What is Abstraction?

Answer:
Abstraction is the process of **hiding implementation details and showing only functionality** to the user.
It reduces complexity and improves modularity.

Achieved using:

- Abstract class
- Interface

---

29 What is an Interface?

Answer:
An interface is a **blueprint of a class**.
It contains **abstract methods** and **static final variables**.
Interface supports **multiple inheritance**.
A class **implements** an interface.

---

30 What is Constructor?

Answer:
Constructor is a **special method** used to initialize class variables.
Constructor name must be same as class name and has **no return type**.
It is automatically called when object is created.

Types:

- Default constructor
- Parameterized constructor

3️⃣1️⃣ What is this keyword?

Answer:
`this` keyword refers to the **current class object**.
It is used to differentiate **instance variables from local variables**.

3️⃣2️⃣ What is static keyword?

Answer:
Static keyword is used for **memory management**.
Static members belong to **class**, not object.
Static methods can access only static data directly.

3️⃣3️⃣ What is final keyword?

Answer:
Final keyword is used to **restrict usage**.

- final variable → value cannot be changed
- final method → cannot be overridden
- final class → cannot be inherited

3️⃣4️⃣ What are Packages in Java?

Answer:
Package is a **collection of related classes and interfaces**.
Types:

- Built-in packages

- User-defined packages

Packages help in **modularization and access control**.

3️⃣5️⃣ What are Access Modifiers?

Answer:
Access modifiers define **scope and visibility** of variables and methods.

- public → accessible everywhere
- protected → package + inheritance
- default → within package
- private → within class only

3️⃣6️⃣ What is Exception?

Answer:
Exception is an **abnormal condition** that disrupts normal program flow.
Java provides **exception handling** to handle runtime errors gracefully.

3️⃣7️⃣ Types of Exception

Answer:
1️⃣ Checked Exception → checked by compiler
Example: IOException, FileNotFoundException

2️⃣ Unchecked Exception → not checked by compiler
Example: NullPointerException, ArithmeticException

3️⃣8️⃣ What is try-catch block?

Answer:
try block contains **risky code**, catch block handles exception.
It prevents abnormal termination of program.

---

39 What is finally block?

Answer:
finally block is used to **close resources**.
It always executes whether exception occurs or not.

---

40 What are Collections?

Answer:
Collections are used to **store and manipulate group of objects**.
Collection is an interface present in **java.util** package.

---

41 What is ArrayList?

Answer:
ArrayList implements List interface.
It allows **duplicate values**, **multiple nulls**, and **preserves insertion order**.

---

42 What is HashSet?

Answer:
HashSet implements Set interface.
It does not allow **duplicates**, allows **only one null**, and **does not preserve order**.

---

43 What is HashMap?

Answer:
HashMap stores data in **key-value pairs**.
Keys are unique, values can be duplicate.
Insertion order is not preserved.

---

44 Why Wrapper Classes are needed?

Answer:
Wrapper classes convert **primitive data into objects**.
Collections work only with **objects**, not primitives.

---

45 What is Autoboxing?

Answer:
Autoboxing is automatic conversion of **primitive to wrapper object**.

---

46 What is Unboxing?

Answer:
Unboxing is automatic conversion of **wrapper object to primitive**.

---

47 What is Type Casting?

Answer:
Type casting is converting **one data type into another compatible type**.

---

48 What is Upcasting?

Answer:
Upcasting is converting **child object reference into parent reference**.
It is implicit and safe.

---

49 What is Downcasting?

Answer:
Downcasting is converting **parent reference into child reference**.
It is explicit and risky if not handled properly.

5️⃣0️⃣ Why Java is important for Testers?

Answer:
Java is widely used in **automation testing**.
Tools like **Selenium, TestNG, Cucumber, JMeter** are Java-based.
Understanding Java helps testers write **automation scripts**, **frameworks**, and debug issues confidently.

5️⃣1️⃣ What is Serialization in Java?

Answer:
Serialization is the process of **converting an object into a byte stream** so that it can be **saved into a file or transferred over network**.
Deserialization is the reverse process of **converting byte stream back into object**.

Why needed:

- To save object state
- To send object from one system to another

In Java, serialization is achieved using **Serializable interface**.

Example:

```
class Student implements Serializable {
    int id;
    String name;
}
```

👉 Serializable is a **marker interface** (no methods).

5️⃣2️⃣ Why Serializable interface does not have any method?

Answer:
Serializable interface is a **marker interface**.
It only informs JVM that **this class objects can be serialized**.
JVM internally handles the serialization process.

5️⃣3️⃣ What happens if a class is not Serializable?

Answer:
If a class does not implement Serializable and we try to serialize its object,
Java throws **NotSerializableException** at runtime.

5️⃣4️⃣ What is transient keyword?

Answer:
`transient` keyword is used to **skip a variable during serialization**.
Transient variables are **not saved** when object is serialized.

Example:

```
transient int password;
```

5️⃣5️⃣ What is Multithreading?

Answer:
Multithreading is a process of **executing multiple threads simultaneously**.
Thread is a **lightweight sub-process**.
Multithreading improves **performance and CPU utilization**.

Real-time example:

- Browser → downloading + scrolling
- Banking app → balance check + transaction

5️⃣6️⃣ What is Thread in Java?

Answer:
Thread is a **smallest unit of execution** in a program.
Java provides built-in support for multithreading using **Thread class**.

5️⃣7️⃣ How to create a Thread in Java?

Answer:
There are **2 ways**:

1) By extending Thread class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}
```

2) By implementing Runnable interface

```
class MyThread implements Runnable {
    public void run() {
        System.out.println("Thread running");
    }
}
```

👉 Runnable is preferred because Java supports **single inheritance**.

5️⃣8️⃣ What is run() method?

Answer:
run() method contains the **logic of thread**.
JVM calls run() when thread starts.

5️⃣9️⃣ Difference between start() and run()?

Answer:

- start() → creates a new thread and then calls run()
- run() → executes like normal method, no new thread created

6️⃣0️⃣ What is synchronization?

Answer:
Synchronization is used to **control access of multiple threads to shared resources**.
It prevents **data inconsistency**.

Without synchronization → wrong output
With synchronization → safe execution

6️⃣1️⃣ What is deadlock?

Answer:
Deadlock occurs when **two or more threads wait forever** for each other's resources.
Program gets stuck.

6️⃣2️⃣ What is Collection time complexity?

Answer:
Time complexity defines **how fast an operation performs**.

Collection	Search	Insert	Delete
ArrayList	O(n)	O(1)	O(n)
HashSet	O(1)	O(1)	O(1)
HashMap	O(1)	O(1)	O(1)

👉 HashMap & HashSet are fast because they use **hashing**.

6️⃣3️⃣ Why ArrayList search is slow?

Answer:
ArrayList stores data in **index-based structure**.
To search an element, it needs to **iterate through elements**, hence O(n).

6️⃣4️⃣ Why HashMap is faster?

Answer:
HashMap uses **hashing technique**.
Key's hashcode is used to directly locate value, so operations are mostly O(1).

6️⃣5️⃣ What is ConcurrentModificationException?

Answer:
This exception occurs when a collection is **modified while iterating** using iterator.

Example:

```
for(String s : list) {
    list.remove(s); // throws exception
}
```

6️⃣6️⃣ What is Iterator?

Answer:
Iterator is used to **traverse collection elements** one by one.
It supports safe removal using `remove()` method.

6️⃣7️⃣ Difference between Array and Collection?

Answer:

- Array → fixed size, supports primitive
- Collection → dynamic size, supports objects only

6️⃣8️⃣ Why collections do not allow primitives?

Answer:
Collections work on **objects**, not primitive data types.
That is why **wrapper classes** are required.

6️⃣9️⃣ What is Garbage Collection?

Answer:
Garbage Collection is the process of **automatically removing unused objects from memory**.
It is handled by **JVM**, programmer does not need to delete objects manually.

7️⃣0️⃣ Why Java is called platform independent?

Answer:
Java code is compiled into **bytecode**, which runs on JVM.
JVM is platform dependent, but bytecode is same.
Hence Java follows **Write Once Run Anywhere**.

Final Interview Tip (For Tester)

As a tester, Java is not about deep development logic.
You should clearly understand:

- OOP concepts
- Collections
- Exception handling
- Multithreading basics
- Wrapper & type casting